

OSTA

Software Requirements Specification (MVP) Manufacturing Outsourcing & Fabrication Operating System

Document	OSTA SRS
Version	1.0
Status	MVP / front-end prototype
Date	2026-09-08
Audience	Developers
Repository	https://github.com/omarmoussa724/osta

Contents

1. Introduction

- 1.1 Purpose
- 1.2 Product scope & vision
- 1.3 Intended audience
- 1.4 Definitions, acronyms, glossary
- 1.5 References

2. Overall description

- 2.1 Product perspective
- 2.2 Product functions (summary)
- 2.3 User classes & characteristics
- 2.4 Operating environment
- 2.5 Design & implementation constraints
- 2.6 Assumptions & dependencies

3. System architecture

- 3.1 Technology stack (as built)
- 3.2 Repository layout
- 3.3 Runtime architecture
- 3.4 Data layer & the backend seam (most important section for a backend dev)
- 3.5 Routing map (from src/App.tsx, hash-prefixed at runtime)
- 3.6 Design system
- 3.7 Build & run

4. Data model

- 4.1 Entity overview
- 4.2 Entity dictionaries
- 4.3 Identity & referential rules
- 4.4 Derived data

5. Functional requirements

- 5.1 Application shell & navigation
- 5.2 Command Center (/)
- 5.3 Projects (/projects)
- 5.4 Project detail (/projects/:code[/:tab])
- 5.5 Assembly view (/projects/:code/asm/:asm)
- 5.6 Part detail (/parts/:id)
- 5.7 Parts (/parts)
- 5.8 Outsourcing Control Center (/outsourcing, /outsourcing/:id)
- 5.9 Production board (/production)
- 5.10 BOM (/bom, and Project > BOM tab)
- 5.11 Materials (/materials)
- 5.12 Nesting (/nesting, /nesting/:id)
- 5.13 QC (/qc)
- 5.14 NCR / Issues (/issues)
- 5.15 CAD/CAM (/cadcam)
- 5.16 Inventory (/inventory)
- 5.17 Suppliers (/suppliers)
- 5.18 Analytics (/analytics)
- 5.19 Settings (/settings)
- 5.20 Global search / command palette
- 5.21 Cross-cutting UI states

6. Business rules

7. External interface requirements

7.1 User interfaces

7.2 Software interfaces (to be built)

7.3 Integrations (future, all optional / additive)

8. Non-functional requirements

9. Roadmap to production (summary)

Appendix A - Route reference

Appendix B - Golden demo path (for smoke testing)

Appendix C - Fabrication glossary

Appendix D - Known MVP limitations

1. Introduction

1.1 Purpose

This document specifies the requirements and the as-built design of **OSTA**, a manufacturing operations application for sheet-metal fabrication and outsourced manufacturing. It is written for a **developer** who needs to understand what the system does, how the current codebase is structured, and what must be built to take it from MVP to a production SaaS.

The MVP delivered in this repository is a **front-end application with in-memory mock data**. There is no backend, authentication, or persistence yet. Section 3.4 and Section 9 describe the seam and the path to a real backend.

1.2 Product scope & vision

OSTA is positioned as *"the operating system for modern fabrication"* - a single operational layer over the workflow:

```
Customer -> Engineering -> BOM -> Materials -> CAD/CAM -> Nesting ->
Outsourcing -> Production -> QC -> Assembly -> Delivery
```

The organising principle is a stable **object hierarchy** in which a part keeps one identity for its whole life:

```
Project -> Assembly -> Part -> Operation -> Material -> Supplier/Factory -> QC
```

In scope for the MVP: a navigable, realistic demonstration of the control workflow - projects, assemblies, parts, BOM, materials, nesting, production, outsourcing and QC - with the OSTA business rules visible in the UI, and a global search that resolves one part number across every module.

Out of scope for the MVP: backend, database, auth, multi-tenancy, real file storage, real integrations (PDM/MES/nesting software), reporting exports, mobile apps, billing. These are the production roadmap (Section 9).

1.3 Intended audience

Primary: engineers implementing the backend and hardening the front end. Secondary: technical founders, prospective contributors, and reviewers.

1.4 Definitions, acronyms, glossary

Term	Meaning
OSTA	Operations System Assistant - the product name
Project	A customer order / job, e.g. PRJ-26001
Assembly	A buildable sub-unit of a project, e.g. ASM-001; may have sub-assemblies (ASM-001.1)
Part	A single manufactured item with a globally unique Part Number , e.g. P-001245
BOM	Bill of Materials - the Project -> Assembly -> Sub-assembly -> Part tree
Operation / Route	The ordered list of process steps for a part (e.g. Laser Cut -> Bend -> Weld -> FINAL_DIM_QC)
Work Order (WO)	Shop-floor authorisation to manufacture; belongs to an Assembly
Production Job	A single operation of a part scheduled on a machine, e.g. PJ-4102
Nesting / Nest	Layout of parts on a raw sheet for cutting, e.g. NEST-0261
Outsource Order	A subcontracted process package, e.g. OUT-0261, requested via an RFQ
RFQ	Request For Quotation sent to a supplier
QC Inspection	A quality check record; types: INCOMING, IN_PROCESS, FINAL_DIM_QC
FINAL_DIM_QC	Final dimensional quality control - the mandatory last gate before a part is "Ready for Assembly"
NCR	Non-Conformance Report - a logged quality/operational defect, e.g. NCR-018
Material grade	Steel specification, e.g. S235JR, S355JR, SS304, SS316
Press brake	Machine that bends sheet metal
EGP	Egyptian Pound - the currency used throughout the seed data
SPA	Single-Page Application

1.5 References

- Repository README.md and osta/README.md - quick start and architecture map.
- marketing/video-kit.md - demo script and the canonical "golden path" click-through.

- Source of truth for the data model: `src/types/index.ts`.
 - Source of truth for routing: `src/App.tsx`.
-

2. Overall description

2.1 Product perspective

Aspect	MVP (this repo)	Target (production)
Delivery	Static SPA (Vite build)	SPA + multi-tenant API + Postgres
Data	In-memory arrays in <code>src/data/*</code>	Database, per-organisation (tenant) rows
Auth	None	Auth provider + RBAC + org membership
Persistence	None (refresh = reset)	Durable, audited, backed up
Integrations	None (import buttons are stubs)	BOM import, PDM, nesting reports, MES, supplier portal
Routing	Hash router (<code>/#/...</code>)	History router with auth guards

The front end is deliberately structured so that **only `src/data/` changes** when a backend is added (Section 3.4).

2.2 Product functions (summary)

1. Present a **Command Center** dashboard: KPIs, a horizontal production-flow view with per-stage counts and delays, an operational-risk feed, an on-time-delivery trend, and an outsourcing snapshot.
2. List and filter **Projects**; open a **Project workspace** with phase progress and seven tabs (Overview, Assemblies, BOM, Parts, Production, Outsourcing, QC).
3. Drill Project -> Assembly -> Part with **breadcrumbs**, each crumb linking to its object.
4. Show a **Part detail** page: specification, a vertical manufacturing timeline with the current operation highlighted, the FINAL_DIM_QC gate, material position, and a "Workflow Connections" panel linking the part to every other module.
5. Provide an **Outsourcing Control Center** and per-job detail with a Released -> Received delivery stepper and action buttons (Send RFQ, Upload Documents, Mark Received).
6. Provide a **Production board** (Kanban: Ready -> In Production -> QC -> Complete -> Blocked) with drag-and-drop and rule enforcement.
7. Provide a **BOM tree**, **Materials** inventory with shortage alerts, **Nesting** list + illustrative sheet layout, **QC** dashboard, **NCR/Issues** register, **CAD/CAM**, **Inventory**, **Suppliers**, **Analytics**, **Settings**.
8. Provide a **global command palette** (Ctrl/Cmd+K) that searches every entity and groups results by module.
9. Show meaningful **empty / loading / blocked** states and **toast** notifications for actions.

2.3 User classes & characteristics

Persona	Goals	Primary screens
Operations / Production Manager	See what is active, delayed, and where the bottleneck is	Command Center, Projects, Production
Production Planner	Sequence work, manage the board, watch due dates	Production, Nesting, Project > Production tab
CAD/CAM Engineer	Track revisions, programmes, machine posting	CAD/CAM, Part detail, Nesting
Procurement Manager	Manage RFQs, suppliers, outsourced delivery, material shortages	Outsourcing, Suppliers, Materials
QC Inspector / Manager	Track inspections, enforce the final gate, manage NCRs	QC, NCR/Issues, Part detail
Engineering Manager	Project health, BOM release status	Projects, Project detail, BOM
Factory Owner / Investor	60-second read of operational health	Command Center, Analytics

2.4 Operating environment

- **Browsers:** current Chrome, Edge, Firefox, Safari (evergreen). Desktop-first.
- **Target resolution:** 1440x900 primary; 1280x800 supported; tablet/mobile simplified (sidebar collapses below 1180 px).
- **Toolchain:** Node.js >= 20 (built with Node 24), npm.
- **No server runtime** for the MVP; the production build is static files.

2.5 Design & implementation constraints

- Stack is fixed: ****React 19 + TypeScript + Vite + Tailwind CSS v3 + React Router v7**
- **lucide-react****. `recharts` is installed but currently unused (charts are hand-drawn SVG); keep or remove deliberately.
- **Dark-first** industrial visual design; a single committed theme (no light mode in the MVP). Tokens in `src/index.css :root`.
- Hash routing so the static build works on any host without rewrite rules.
- `tsconfig.app.json` has `verbatimModuleSyntax: true` (use `import type`), `erasableSyntaxOnly: true` (no enums / no TS parameter properties). `noUnusedLocals` / `noUnusedParameters` were relaxed to `false` for MVP velocity - re-enable before production.
- No secrets in the repo. The mock build must remain the sales demo (never demo on real customer data).

2.6 Assumptions & dependencies

- All monetary values are **EGP**; weights are **metric tonnes / kg**; the reference "today" for relative dates is **2026-09-07** (`TODAY` in `src/lib/format.ts`).
- Seed data models an Egyptian fabrication shop (customers, suppliers, machines).
- External libraries are pinned via `package-lock.json`.

3. System architecture

3.1 Technology stack (as built)

Layer	Choice	Version
UI runtime	React / React DOM	19.2
Language	TypeScript	~6.0
Build / dev server	Vite	8.2
Styling	Tailwind CSS v3 + hand-written component CSS	3.4
Routing	react-router-dom (HashRouter)	7.18
Icons	lucide-react (wrapped by components/Icon.tsx)	1.42
Class utility	clsx (via lib/cn.ts)	2.1
Charts	Inline SVG (components/charts.tsx); recharts installed, unused	3.10
Lint	oxlint	1.79

No state-management library - React useState / useContext only. No data-fetching library yet (add TanStack Query with the backend).

3.2 Repository layout

```
osta/
index.html      # Vite entry; loads Google Fonts (Archivo/Inter/IBM Plex Mono)
src/
  main.tsx      # createRoot + <HashRouter><App/></HashRouter>
  App.tsx       # <ToastProvider> + <Routes> (see 3.5)
  index.css     # design tokens (:root) + all component CSS + responsive rules
  types/index.ts # ALL domain interfaces (Section 4)
  lib/
    cn.ts       # cn(), hexA(), mulberry32()
    format.ts   # TODAY, fmtInt, fmtEGP, fmtT, fmtDate, fmtDateShort, daysFromToday
    status.ts   # MFG, PROJ_STATUS, OUT_STATUS, OUT_STEPS, TONE_HEX, toneColor()
  data/
    projects.ts assemblies.ts parts.ts materials.ts machines.ts
    suppliers.ts outsource.ts jobs.ts qc.ts issues.ts nesting.ts
    misc.ts     # FLOW, RISKS, KPIS, ONTIME_TREND, OUT_SPEND, EXPOSURE_BY_GRADE,
                # STOCK, MOVES, RULES
    index.ts    # re-exports + selectors + buildSearchIndex() + connectionsFor()
  components/
    Icon.tsx    # string name -> lucide component map
    ManufacturingStatus.tsx # <Pill>, <MfgStatus>, <ProjStatus> (the required shared status component)
    ui.tsx      # <Panel> <Bar> <Empty> <PageHead> <Crumbs>
    charts.tsx  # <Spark> <BarSH> <AreaTrend>
    tables.tsx  # <PartsTable> <OutsourceTable> <QcTable>
    FlowStages.tsx # dashboard production-flow strip
    SheetView.tsx # illustrative nesting sheet (deterministic shelf pack)
    BomTree.tsx  # collapsible BOM tree (used by Bom page and Project>BOM tab)
    CommandPalette.tsx # Ctrl/Cmd+K global search overlay
  layout/
    AppLayout.tsx # .app grid: <Sidebar> + <main>(<Topbar/> + <Outlet/>) + <CommandPalette>
    Sidebar.tsx  # grouped nav (Command Center / Engineering / Materials / Quality / System)
    Topbar.tsx   # (merged into AppLayout) breadcrumbs + date + search + bell + avatar
    ToastProvider.tsx # context + useToast() + toast viewport
    crumbs.ts    # resolveCrumbs(pathname) -> Crumb[]
  pages/         # one file per screen (Section 5)
docs/           # this document
marketing/      # video kit
osta-mvp.html (repo root as osta-mvp.single-file.html) # zero-toolchain CDN build of the same app
```

3.3 Runtime architecture

- `main.tsx` mounts `<HashRouter><App/></HashRouter>`.
- `App.tsx` wraps everything in `<ToastProvider>` and declares a single layout route `<AppLayout>` with all pages as children; `<Outlet/>` renders the active page.
- `AppLayout` owns: the sidebar, the top bar (breadcrumbs via `resolveCrumbs`, date, search trigger, notifications, avatar), the `Ctrl/Cmd+K` key handler, the mobile nav toggle, and mounts `<CommandPalette>`.
- Pages read data **synchronously** from `src/data` selectors and render. There is no loading state today because data is in memory; the components are written so a `loading` branch can be added when selectors become async.
- Local, ephemeral UI state only (filters, board columns, palette query). The Production board keeps a local `useState` copy of jobs so drag-and-drop mutations are visible but not persisted.

3.4 Data layer & the backend seam (*most important section for a backend dev*)

Every page imports from `src/data/index.ts` and never from a network client. `index.ts` today:

1. `export *` from each entity file (the raw arrays), and
2. defines **selector functions** - `findProject(code)`, `findPart(id)`, `assemblyParts(code)`, `projectParts(code)`, `findOut(id)`, `findNest(id)`, `findSupplier(id)`, `findMat(grade, thk)`, `findMachine(name)`, `findIssue(id)`, `buildSearchIndex()`, `connectionsFor(partId)`.

To attach a backend: replace the bodies of these selectors with API calls and change their return types to `Promise<T>` (or wrap them in TanStack Query hooks such as `useProject(code)`), then update pages to handle the pending/error states. No component's JSX structure needs to change - they already consume selectors, not imports of raw arrays. Recommended concrete steps:

- Add `src/lib/api.ts` (typed client) and `src/data/queries.ts` (hooks).
- Keep `src/types/index.ts` as the shared contract between client and server (or generate it from the API schema).
- Move the business rules (Section 6) server-side as invariants so the API cannot be used to bypass them.

3.5 Routing map (from `src/App.tsx`, hash-prefixed at runtime)

Path	Component	Notes
/	Dashboard	index route
/projects	Projects	filterable table
/projects/:code	ProjectDetail	default tab = Overview
/projects/:code/:tab	ProjectDetail	tab in {overview, assemblies, bom, parts, production, outsourcing, qc}
/projects/:code/asm/:asm	AssemblyView	assembly detail (declared before :code/:tab so asm wins)
/parts	Parts	global parts table + filters
/parts/:id	PartDetail	part lifecycle + connections
/production	Production	Kanban board
/outsourcing	Outsourcing	control-center table + KPIs
/outsourcing/:id	OutsourcingDetail	accepts OUT-xxxx or RFQ-xxxx
/bom	Bom	all-projects BOM tree
/cadcam	CadCam	programmes list + model/revision panel
/nesting	Nesting	nest cards with sheet previews
/nesting/:id	NestingDetail	full sheet layout + nest data
/materials	Materials	inventory + shortage alerts
/inventory	Inventory	stock by rack/lot + movements
/suppliers	Suppliers	supplier cards
/qc	Qc	QC dashboard + inspections
/issues	Issues	NCR / issues register
/analytics	Analytics	trend + WIP + spend + exposure charts
/settings	Settings	org profile + rules + integrations (stubs)
*	NotFound	empty state -> back to Command Center

3.6 Design system

- **Tokens** (`src/index.css :root`): ground #0B0D10, panels #14171C/#181C22/ #1E232B, hairlines #252A32, ink #E7EAE6/#A2ABB6/#6D7681, accent #4C9AFF. Semantic: --ok #35C88F, --warn #E0A63C, --crit #E5484D, --violet #9A7CF0 (outsourced), --idle #6D7681.
- **Type**: Archivo (display/headings), Inter (body), IBM Plex Mono (part numbers, codes, all figures / tabular-nums).
- **Shared status component (required)**: `components/ManufacturingStatus.tsx` exports `<Pill tone>`, `<MfgStatus s>` and `<ProjStatus s>`. MfgStatus states: NOT_STARTED, READY, IN_PRODUCTION, BLOCKED, QC, PASSED, FAILED, OUTSOURCED, COMPLETE - one consistent treatment everywhere.
- **Layout primitives**: `.app` (240px sidebar grid), `.panel`, `.tbl`, `.kpi`, `.bar`, `.pill`, `.btn`, `.board/.klane/.jcard`, `.tl` (timeline), `.flow/.stage`, `.cmdk*`, `.toast*` - all in `src/index.css`.

3.7 Build & run

```
npm install
npm run dev      # http://localhost:5173
npm run build    # tsc -b && vite build -> dist/ (verified clean)
npm run preview  # serve dist/ locally
npm run lint     # oxlint
```

The single-file `osta-mvp.single-file.html` is an independent build (React via CDN, Babel-in-browser). It is kept in sync manually and is only for zero-setup demos.

4. Data model

4.1 Entity overview

```
Organization (future) 1---* Project
Project 1---* Assembly      Assembly 0..1 parent Assembly (sub-assemblies)
Assembly 1---* Part         Part.id is GLOBALLY UNIQUE (the Part Number)
Part *---1 Material (by grade+thk)
Part 1---* Operation (ordered string route incl. FINAL_DIM_QC)
Part 1---* ProductionJob     ProductionJob *---1 Machine
Part 0..* OutsourceOrder (part listed in order.parts[]) OutsourceOrder *---1 Supplier, *---1 Project
Part 1---* QCInspection
Part | Assembly | Supplier | Material 1---* Issue (NCR, polymorphic ref)
Nest groups parts by (grade, thk, project)
```

4.2 Entity dictionaries

Source of truth: `src/types/index.ts`. Types below are TypeScript.

Project

Field	Type	Notes
code	string	PK, PRJ-NNNNN
name	string	
customer	string	
start, due	string	ISO YYYY-MM-DD
progress	number	0-100 headline %
status	ProjStatusKey	ENGINEERING MATERIAL NESTING IN_PRODUCTION OUTSOURCED QC ASSEMBLY DELIVERY DELAYED
phases	Record	Engineering/Material/Nesting/Fabrication/Assembly/QC/D each 0-100

Assembly

Field	Type	Notes
code	string	PK, ASM - NNN or ASM - NNN . M for sub-assemblies
name	string	
project	string	FK -> Project.code
parent	string null	FK -> Assembly.code (sub-assembly)
completion	number	0-100
currentOp	string	human label

Part (the identity object)

Field	Type	Notes
id	string	PK, globally unique Part Number P-NNNNNN
desc	string	
project	string	FK -> Project.code
assembly	string	FK -> Assembly.code
grade	string	material grade
thk	number	thickness, mm
qty	number	quantity for the assembly
unitKg, weightKg	number	unit and total mass
rev	string	drawing revision (A, B, C, ...)
drawing	string	drawing number
cad	string	CAD file name
ops	string[]	ordered route; always contains FINAL_DIM_QC , ends with Ready for Assembly
opIndex	number	index into ops of the current step
status	MfgStatusKey	NOT_STARTED READY IN_PRODUCTION BLOCKED QC PASSED FAILED OUTSOURCED COMPLETE
machine	string null	current machine name
outsource	string null	FK -> OutsourceOrder.id
qcState	'NONE' 'PENDING' 'PASSED' 'FAILED'	FINAL_DIM_QC result
note	string null	blocker / context text
matKey	string null	derived grade - thk key into Material

Material

grade, thk, family, avail, reserved, required, price (EGP/tonne); derived key = grade-thk and shortage = $\max(0, \text{required} - (\text{avail} - \text{reserved}))$.

Machine

id, name, group (Laser Cutting / Punching / Press Brake / CNC Turning / CNC Milling / Welding / Quality), util %, queue (parts).

Supplier

id (SUP-NN), name, city, processes[], rating (0-5), onTime %, capacity %, contact, phone, email.

OutsourceOrder

Field	Type	Notes
id	string	PK OUT-NNNN
rfq	string	RFQ-NNNN (alt lookup key)
supplier	string	FK -> Supplier.id
project	string	FK -> Project.code
process	string	Laser Cutting / CNC Machining / Press Brake / Welding / Painting / Powder Coating / Hot-Dip Galvanizing / Plate Rolling
partsCount	number	headline count
parts	string[]	FK -> Part.id (itemised where known)
weightT	number	tonnes
value	number	quoted EGP
actual	number null	actual EGP
due	string	need date
sent	string	RFQ sent date
eta	string null	expected delivery
status	OutStatusKey	RFQ_SENT RELEASED MATERIAL_CONFIRMED PRODUCTION QC DISPATCH RECEIVED AT_RISK
note	string?	risk / status context

Delivery step order: OUT_STEPS = [RELEASED, MATERIAL_CONFIRMED, PRODUCTION, QC, DISPATCH, RECEIVED];
OUT_STATUS[status].step maps a status onto that axis (AT_RISK -> step 2, RFQ_SENT -> -1).

ProductionJob

id (PJ-NNNN), part (FK), op, machine, operator, qty, due, progress 0-100, col ('READY' | 'IN_PRODUCTION' | 'QC' | 'COMPLETE' | 'BLOCKED'), block? (reason string).

QCInspection

id (QC-NNNN), part (FK), type ('INCOMING'|'IN_PROCESS'|'FINAL_DIM_QC'), result ('PASSED'|'FAILED'|'PENDING'), date, inspector, note.

Issue (NCR)

id (NCR-NNN), ref (Part.id | Assembly.code | Supplier.id | Material.key), refType ('part'|'assembly'|'supplier'|'material'), severity ('Critical'|'Major'|'Minor'), status ('Open'|'In Rework'|'Closed'), opened, owner, title, assembly?.

Nest

id (NEST-NNNN), project (FK), grade, thk, sheetW, sheetL (mm), parts (count), util (%), machine, status ('READY'|'RUNNING'|'QUEUED'|'BELOW_TARGET'). NEST_TARGET = 80 (%).

Support datasets (data/misc.ts)

FLOW (8 stages: key, count, delayed, status), RISKS, KPIS, ONTIME_TREND, OUT_SPEND, EXPOSURE_BY_GRADE, STOCK, MOVES, RULES.

4.3 Identity & referential rules

- **Part Number is unique and immutable.** The same P-xxxxxx string is the join key across BOM, Nesting, Production, Outsourcing and QC. Enforce as a unique constraint per organisation in the database.
- **Manufacturing context = Project code + Assembly code.** A Work Order is scoped to exactly one (project, assembly) pair.
- All *_id / *_code fields listed as FK must be enforced as foreign keys server-side.

4.4 Derived data

- buildSearchIndex() -> flat SearchEntry[] over Projects, Assemblies, Parts, OutsourceOrders, Nests, Materials, Suppliers, Issues (used by the command palette).
- connectionsFor(partId) -> Connection[]: for a given part it assembles links to its Project, Assembly, BOM line, Material, Nest, Production Job, Outsource Order, QC inspections and Issues. This function encodes the "one part, every module" principle and should be reproduced as a backend endpoint (GET /parts/:id/connections).
- KPIs and risk cards in the MVP are partly curated constants; in production they are computed queries.

5. Functional requirements

Notation: **FR-..** "Shall" = required. Acceptance criteria are testable. Where a requirement is only partially met by the MVP it is marked (*MVP: partial*).

5.1 Application shell & navigation

- **FR-SHELL.1** The app shall present a persistent left sidebar with the OSTA logo, five grouped sections (Command Center, Engineering, Materials, Quality, System) and the footer block "OSTA MVP / Operations System Assistant".
- **FR-SHELL.2** The active route shall be highlighted in the sidebar.
- **FR-SHELL.3** A top bar shall show breadcrumbs, the current date, a search trigger (with `Cmd/Ctrl+K` hint), a notifications icon, and a user avatar.
- **FR-SHELL.4** Breadcrumbs shall be generated from the current path and every crumb except the last shall be a link to its object (`resolveCrumbs` in `layout/crumbs.ts`). Example: OSTA / Projects / PRJ-26001 / ASM-001.1 / P-001245.
- **FR-SHELL.5** Below 1180 px the sidebar shall collapse and be toggled by a menu button in the top bar.
- **FR-SHELL.6** Actions that mutate or would call a backend shall surface a **toast** (`useToast`); toasts auto-dismiss after ~3.6 s.

5.2 Command Center (/)

- **FR-DASH.1** Display six KPI cards: Active Projects, Parts in Production, Outsourced Parts, Material Exposure (EGP), At Risk, On-Time Delivery - each with a trend direction and a sparkline.
- **FR-DASH.2** Display the **Production Flow** strip: eight stages (Engineering, Material, Nesting, Cutting, Bending, Welding, QC, Assembly), each showing count, delayed count and a status tint; the bottleneck stage (`status = 'crit'`) shall be visually emphasised.
- **FR-DASH.3** Display an **Operational Risks** list; each row shows a severity dot, title, meta line and a tag, and links to the relevant object.
- **FR-DASH.4** Display an **On-Time Delivery** area chart with a target line and the **Outsourcing Snapshot** counts by state.
- **FR-DASH.5** Display the first six projects in a table; a row click opens the project.
- **FR-DASH.6** A first-time user shall be able to answer "what's active / outsourced / in production / delayed / where's the bottleneck / what's at risk" within ~60 s from this page alone.

5.3 Projects (/projects)

- **FR-PROJ.1** List all projects with columns: Project (code + name), Customer, Progress bar, Assemblies, Parts, Material weight, Outsourced value, Delivery (date + days remaining/late), Status.
- **FR-PROJ.2** Provide quick filters: All, In Production, Material, Quality, Assembly, At Risk (`status in {MATERIAL, DELAYED}` OR due within 10 days).
- **FR-PROJ.3** A row click navigates to `/projects/:code`.

5.4 Project detail (/projects/:code[:tab])

- **FR-PDET.1** Header shows code, name, customer, start, delivery, days remaining/late, status pill, and a "Status pack" action (*MVP: toast stub*).
- **FR-PDET.2** "Project Progress" panel shows the seven phase bars from `project.phases`.

- **FR-PDET.3** "At a glance" panel shows assembly count, part count, material weight, outsourced value, and open NCR count (computed from issues referencing the project's assemblies).
- **FR-PDET.4** Tab bar: Overview, Assemblies, BOM, Parts, Production, Outsourcing, QC. The tab is the :tab path segment; unknown/absent = Overview.
- **FR-PDET.5 Overview** shows an assemblies mini-table and a recent-activity feed.
- **FR-PDET.6 Assemblies** lists top-level assemblies with parts/weight/completion/ current-op and nested sub-assemblies; each links to the assembly view. If none: a meaningful empty state with an "Import assemblies" stub.
- **FR-PDET.7 BOM** renders the BomTree scoped to this project.
- **FR-PDET.8 Parts** renders PartsTable for the project's parts (or empty state).
- **FR-PDET.9 Production** lists the project's production jobs (or empty state).
- **FR-PDET.10 Outsourcing** renders outsourceTable for the project (or empty state).
- **FR-PDET.11 QC** renders qcTable for inspections of the project's parts (or empty state).

5.5 Assembly view (/projects/:code/asm/:asm)

- **FR-ASM.1** Header shows assembly code + name, parent project, and (if a sub-assembly) its parent assembly.
- **FR-ASM.2** Five metric cards: Parts, Material Weight, Completion, Current Operation, Outsourced (n / total).
- **FR-ASM.3** Parts table: Part No, Description (+ rev, drawing), Material, Thickness, Qty, Operation route (mid-route steps), Status. Row click opens the part.
- **FR-ASM.4** A chip shall show how many parts are "ready for assembly" (status = READY).

5.6 Part detail (/parts/:id)

- **FR-PART.1** Header: part number, description, links to project and assembly, revision, a drawing action (stub), and the current status pill.
- **FR-PART.2** If status in {BLOCKED, FAILED} show a prominent red banner with part.note.
- **FR-PART.3 Manufacturing Timeline:** render part.ops as a vertical stepper; steps before opIndex are "done", opIndex is "current" (highlighted, shows the machine), a failed FINAL_DIM_QC renders as "failed".
- **FR-PART.4** The FINAL_DIM_QC step shall be labelled "mandatory gate". The final step "Ready for Assembly" shall show "gated - FINAL_DIM_QC must pass first" whenever qcState != 'PASSED' (see BR-5).
- **FR-PART.5 Workflow Connections** panel: render connectionsFor(id) as a grid of links (Project, Assembly, BOM, Material, Nesting, Production, Outsourcing, QC, Issues), each navigating to that module.
- **FR-PART.6 Specification** panel: project, assembly, material, thickness, quantity, unit & total weight, revision, drawing, CAD file, current status.
- **FR-PART.7 Final Gate** panel: show FINAL_DIM_QC state; show "Ready for Assembly" as **Released** only if qcState = 'PASSED', otherwise **Locked**.
- **FR-PART.8 Material Position** panel (if the grade/thk is known): available, required, shortage (red if > 0).

5.7 Parts (/parts)

- **FR-PLIST.1** Global table of all parts with free-text search (id, desc, grade, project, assembly) and a status filter (All, In Production, Outsourced, Ready, Blocked, QC Failed, Not Started).

5.8 Outsourcing Control Center (/outsourcing, /outsourcing/:id)

- **FR-OUT.1** KPI row: Active Jobs, Outsourced Value (EGP), Weight Outsourced (t), At Risk count.

- **FR-OUT.2** Filter: Active (status != RECEIVED), At Risk (AT_RISK or due < 3 days), All. Table columns: RFQ (+ OUT id), Supplier (+ city), Project, Process, Parts, Weight, Value, Due (+ days), Status.
- **FR-OUT.3** Empty state text shall be specific ("No active outsourcing jobs..."), never a generic "No data".
- **FR-OUTD.1** Detail header: OUT id, process, supplier, project link, RFQ number.
- **FR-OUTD.2** Action buttons: **Send RFQ**, **Upload Documents**, **Mark Received** - each fires a toast (MVP: stubs).
- **FR-OUTD.3 Job Progress** stepper across OUT_STEPS with the current step from OUT_STATUS[status].step; AT_RISK/note shown as a coloured banner.
- **FR-OUTD.4** Panels: Scope (itemised parts where known, else bulk count), Documents (stub list with download toasts), Supplier (contact card), Commercial (process, parts, weight, quoted vs actual cost, sent/expected/need dates, delivery state).

5.9 Production board (/production)

- **FR-PRODB.1** Five columns: Ready, In Production, QC, Complete, Blocked. Each card shows part number + job id, description, operation, machine, operator, qty, due, and a progress bar (or the block reason for blocked cards).
- **FR-PRODB.2** Cards shall be draggable between columns (HTML5 DnD); the drop target highlights.
- **FR-PRODB.3** Moving a card whose part has qcState = 'FAILED' into **Complete** shall be **rejected** with a toast ("FINAL_DIM_QC must pass..."). (BR-4/BR-5.)
- **FR-PRODB.4** A card click opens the part.
- **FR-PRODB.5** Board mutations are local-only in the MVP (not persisted).

5.10 BOM (/bom, and Project > BOM tab)

- **FR-BOM.1** Render a collapsible tree: Project -> Assembly -> Sub-assembly -> Part.
- **FR-BOM.2** Each part row shows qty, grade, thickness, revision and its MfgStatus pill (status rolls up visually).
- **FR-BOM.3** Assembly and part codes in the tree are links to their pages; clicking a link shall not toggle the row.
- **FR-BOM.4** An "Import BOM" button shall be present (MVP: toast stub).

5.11 Materials (/materials)

- **FR-MAT.1** KPI row: grades tracked, shortage lines, total shortage (t), reserved exposure (EGP).
- **FR-MAT.2** For every material with shortage > 0 render a red alert card with required / available / shortage and the EGP value of the shortage. (BR-6.)
- **FR-MAT.3** Inventory table: Material, Family, Thickness, Available, Reserved, Required, Shortage, Coverage bar ((avail-reserved)/required).

5.12 Nesting (/nesting, /nesting/:id)

- **FR-NEST.1** Grid of nest cards: id, status pill, an SVG sheet preview, grade/thickness, part count, utilisation bar vs NEST_TARGET (80%), machine, scrap %.
- **FR-NEST.2** Detail: large illustrative sheet layout (SheetView, deterministic shelf-pack - explicitly not a real nesting result), a Nest Data panel, a below-target warning when util < NEST_TARGET, and a table of parts on that grade/thickness for the project.

5.13 QC (/qc)

- **FR-QC.1** KPI row: Pending Inspections, Passed (30 d), Failed (30 d), Open NCRs, In Rework.
- **FR-QC.2** Filter (All, Pending, Passed, Failed) and an inspections table (qcTable): Inspection, Part, Type (FINAL_DIM_QC marked with *), Result pill, Date, Inspector, Note. Row click opens the part.

5.14 NCR / Issues (/issues)

- **FR-NCR.1** Register table: NCR id, Against (ref), Title, Severity pill, Status pill, Owner, Opened. Filter: Open, Closed, All.
- **FR-NCR.2** A row links to the referenced object (part / supplier / material / assembly) based on refType.
- **FR-NCR.3** A "Raise NCR" button shall be present (MVP: stub).

5.15 CAD/CAM (/cadcam)

- **FR-CAD.1** Programmes table: Part, CAD file, Revision, CAM status (Not programmed / Programmed / Posted - derived from part progress in the MVP), Machine.
- **FR-CAD.2** Right panel: an isometric wireframe placeholder for the selected part and a revision history stepper.

5.16 Inventory (/inventory)

- **FR-INV.1** Stock-on-hand table by Material, Heat/Lot, Location (rack), Qty (kg), State (Available / Reserved / Quarantine).
- **FR-INV.2** Recent movements list (Received / Issued / Reserved / Quarantine) with signed quantities and references.

5.17 Suppliers (/suppliers)

- **FR-SUP.1** Supplier cards: name, city, processes, star rating, on-time %, active job count, capacity/load bar.

5.18 Analytics (/analytics)

- **FR-ANA.1** Four charts (hand-drawn SVG): On-Time Delivery 12-week trend with target; WIP by stage (from FLOW, bottleneck highlighted); Outsourcing spend by process; Material exposure by grade.

5.19 Settings (/settings)

- **FR-SET.1** Organisation panel (workspace, company, location, units = metric, currency = EGP, fiscal year).
- **FR-SET.2** "Manufacturing Rules" panel listing the seven OSTA rules (Section 6), each shown as **Enforced**.
- **FR-SET.3** Integrations panel (ERP, PDM, MES, MRP) with "Connect" buttons (MVP: "coming soon" toasts).

5.20 Global search / command palette

- **FR-SEARCH.1** Cmd/Ctrl+K toggles a modal search overlay; Esc closes it.
- **FR-SEARCH.2** Typing filters SEARCH_INDEX (substring over label + sub + type + code); results are grouped by entity type; Up/Down move the selection, Enter navigates, a row click navigates.
- **FR-SEARCH.3** Searching a part number (e.g. P-001245) shall surface that part and its Project, Assembly, Outsourcing, Nesting and Material entries - demonstrating that one identifier connects the whole workflow.

5.21 Cross-cutting UI states

- **FR-STATE.1** Every list/table has a specific **empty state** (icon + title + hint), never a bare "No data".
 - **FR-STATE.2** Blocked/failed objects show a red banner with the reason.
 - **FR-STATE.3** Actions show success/info via toasts; destructive/irreversible actions are not present in the MVP.
 - **FR-STATE.4** (*MVP gap*) No `loading` or `error` states because data is synchronous; these must be added with the backend.
-

6. Business rules

These are the defining rules of OSTA. In the MVP they are enforced in the UI; in production they **must be enforced in the backend** as invariants.

ID	Rule	MVP enforcement	Production requirement
BR-1	A Work Order belongs to exactly one Assembly (context = Project code + Assembly code).	Data model; assembly views scope everything by <code>(project, assembly)</code> .	FK + WO cannot be created without an assembly.
BR-2	A Work Order may exist only when its Assembly contains ≥ 1 released Part.	Assemblies tab shows "WO required - blocked" when an assembly has no parts.	API rejects WO creation for a part-less assembly.
BR-3	A Part Number is identical across BOM, Nesting, Production, Outsourcing and QC.	Single <code>Part.id</code> used as the join key; <code>connectionsFor()</code> proves it; command palette resolves it.	Unique constraint per org; all modules reference <code>part_id</code> .
BR-4	Final dimensional QC (<code>FINAL_DIM_QC</code>) is a mandatory step on every part route.	Every seed part's <code>ops</code> array contains <code>FINAL_DIM_QC</code> ; Part detail labels it "mandatory gate".	Route templates must include it; validation on route save.
BR-5	A Part cannot become "Ready for Assembly" until <code>FINAL_DIM_QC</code> passes.	Part timeline shows the final step "gated"; Final Gate panel shows Locked unless <code>qcState = 'PASSED'</code> ; the Production board refuses to move a QC-failed job to Complete .	State machine: PASSED (final QC) is a precondition of READY ; API rejects the transition otherwise.
BR-6	Material shortages surface automatically as operational risks.	<code>Material.shortage = max(0, required - (avail - reserved))</code> ; Materials page renders alert cards; dashboard risk feed lists shortages.	Computed on write; raises a Risk/alert record and notifies procurement.
BR-7	Every outsourced job has an explicit delivery state.	<code>OutsourceOrder.status</code> on the RELEASED -> RECEIVED axis; detail stepper.	Enforced enum + transition log per order.

7. External interface requirements

7.1 User interfaces

Desktop-first SPA, dark theme, keyboard: `Cmd/Ctrl+K` (search), `Esc` (close overlay), `Up/Down/Enter` (palette). All navigation is client-side.

7.2 Software interfaces (to be built)

The backend contract should mirror the current selectors:

Selector (client, now)	Proposed endpoint
PROJECTS, <code>findProject(code)</code>	<code>GET /projects</code> , <code>GET /projects/:code</code>
<code>projectAssemblies</code> , <code>findAssembly</code>	<code>GET /projects/:code/assemblies</code> , <code>GET /assemblies/:code</code>
<code>projectParts</code> , <code>assemblyParts</code> , <code>findPart</code>	<code>GET /parts?project=, ?assembly=</code> , <code>GET /parts/:id</code>
<code>connectionsFor(id)</code>	<code>GET /parts/:id/connections</code>
OUTSOURCE, <code>findOut</code>	<code>GET /outsource-orders</code> , <code>GET /outsource-orders/:id</code>
<code>JOBS</code> + board moves	<code>GET /production-jobs</code> , <code>PATCH /production-jobs/:id (column)</code>
QC, ISSUES, MATERIALS, NESTS, SUPPLIERS, MACHINES	<code>GET /qc</code> , <code>/issues</code> , <code>/materials</code> , <code>/nests</code> , <code>/suppliers</code> , <code>/machines</code>
<code>buildSearchIndex()</code>	<code>GET /search?q=</code>

Recommended: JSON REST or tRPC; TanStack Query on the client; `src/types/index.ts` shared or generated as the DTO contract. Every request is tenant-scoped.

7.3 Integrations (future, all optional / additive)

- **BOM import** - Excel/CSV upload with column mapping (the "Import BOM" button).
 - **CAD / PDM** - attach STEP/DXF/PDF to parts; later SolidWorks PDM / Autodesk Vault.
 - **Nesting** - import nest reports from TRUMPF TruTops / Lantek / SigmaNEST / Bystronic.
 - **MES / machines** - production status via OPC-UA / MTConnect / vendor MES.
 - **Supplier portal** - scoped external logins so subcontractors update OUT-xxxx state, dates and documents themselves.
 - Manual entry must always remain possible.
-

8. Non-functional requirements

ID	Requirement
NFR-PERF.1	First contentful paint < 1.5 s on a mid laptop over broadband; route changes < 100 ms (client-only). Production build JS is ~360 kB / ~105 kB gzip today.
NFR-PERF.2	Tables render up to a few hundred rows without virtualisation; add windowing when a real dataset exceeds ~1k rows.
NFR-BROWSER.1	Support the last 2 versions of Chrome, Edge, Firefox, Safari.
NFR-RESP.1	Fully usable at 1440x900 and 1280x800; degrade gracefully to tablet (sidebar collapses < 1180 px); mobile is simplified, not a target.
NFR-A11Y.1	(MVP gap) Needs a pass: focus-visible styles, ARIA on the palette/board/dialogs, keyboard DnD alternative, contrast audit (dark theme already high-contrast), prefers-reduced-motion is already respected.
NFR-SEC.1	(future) Auth provider; RBAC by persona; per-tenant row-level security; audit log on every state change; no secrets in the repo; HTTPS only.
NFR-SEC.2	Business rules (Section 6) enforced server-side, not only in the UI.
NFR-MAINT.1	TypeScript strict; re-enable noUnusedLocals/noUnusedParameters; npm run lint (oxlint) clean; one component/concept per file; shared UI only via components/.
NFR-MAINT.2	Domain types live in src/types; data access only via src/data (or its successor src/lib/api.ts). Pages never call fetch directly.
NFR-OBS.1	(future) Error tracking (e.g. Sentry), structured logs, uptime + daily DB backup with a tested restore.
NFR-I18N.1	Currency is EGP and dates are en-GB formatted via lib/format.ts; centralise here before adding locales.
NFR-LICENSE.1	Confirm licences of all deps (all MIT today) before commercial release; add a LICENSE file.

9. Roadmap to production (summary)

Detail is in the project conversation / ROADMAP notes; condensed:

1. **Wedge:** lead with Outsourcing control + part traceability; everything else stays light.
 2. **Backend:** Postgres + Prisma + typed API; schema from `src/types`; `org_id` + row-level security from day one; move BR-1...BR-7 server-side.
 3. **Auth & tenancy:** auth provider, roles per persona, HashRouter -> BrowserRouter with guards, deploy (static FE + managed Postgres + Sentry + backups).
 4. **Ingestion (the adoption blocker):** real BOM Excel import, drawing upload, nesting-report import, manual production updates; every integration optional.
 5. **Stickiness:** comments/activity feed, notifications, audit log, supplier portal, PDF status pack.
 6. **Pilots:** 2-3 design-partner factories; success = the daily production meeting runs on OSTA instead of a spreadsheet.
-

Appendix A - Route reference

See Section 3.5. Runtime URLs are hash-prefixed, e.g. `http://localhost:5173/#/parts/P-001245`.

Appendix B - Golden demo path (for smoke testing)

Dashboard -> open **PRJ-26001** (Industrial Conveyor) -> **Assemblies** tab -> **ASM-001** -> part **P-001245** (Side Plate, S235JR 6 mm) - shown **Blocked** at "Material Reserved", 1.7 t short -> from Part detail open **Materials** (S235JR 6 mm shortage) -> open **NEST-0261** -> **Production** board -> **OUT-0261** (Cairo Precision Fabrication, at risk) -> **QC** -> confirm "Ready for Assembly" is **Locked** until `FINAL_DIM_QC` passes. Then `Cmd/Ctrl+K` -> type `P-001245` -> results span Project, Assembly, Nesting, Outsourcing, Material.

Key seed figures: 12 active projects, 486 parts in production, 74 outsourced, EGP 1.84M material exposure, 18 at risk, 87% on-time; Cutting is the bottleneck (142 in queue, 18 delayed).

Appendix C - Fabrication glossary

Laser cutting / punching - sheet cutting processes. **Press brake / bending** - forming sheet into angles. **Welding** - MIG/TIG joining. **Plate rolling** - forming curved sections. **Galvanizing / powder coating / painting** - surface finishing, usually outsourced. **Nesting** - arranging parts on a stock sheet to minimise scrap; utilisation = used area / sheet area. **S235JR / S355JR** - structural steel grades; **SS304 / SS316** - stainless grades. **Heat / lot** - a traceable batch of raw material with a mill certificate. **FINAL_DIM_QC** - final dimensional inspection, the OSTA release gate.

Appendix D - Known MVP limitations

- No persistence, auth, or multi-tenancy; refresh resets all state.
- Import / RFQ / "status pack" / integration buttons are toast stubs.
- KPIs and some risk cards are curated constants, not computed.
- SheetView nesting is illustrative only (deterministic shelf-pack, not a nester).
- `recharts` is a dependency but unused (charts are inline SVG).
- No automated tests yet; `noUnusedLocals/noUnusedParameters` disabled.
- Accessibility and `loading/error` states need a dedicated pass.